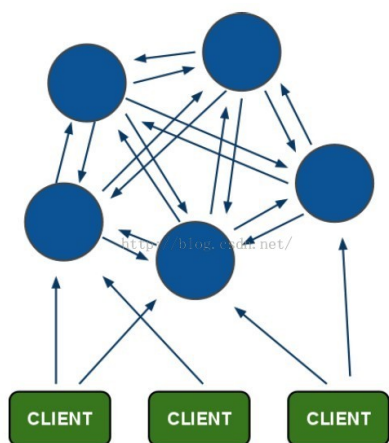


综合业务管理平台分布式redis集群

开发与运维

马杰

1. Redis Cluster的架构图。



(1)所有的redis节点彼此互联(PING-PONG机制),内部使用二进制协议优化传输速度和带宽.

(2)节点的fail是通过集群中超过半数的节点检测失效时才生效.

(3)客户端与redis节点直连,不需要中间proxy层.客户端不需要连接集群所有节点,连接集群中任何一个可用节点即可。

(4)redis-cluster把所有的物理节点映射到[0-16383]slot上(哈希槽),cluster 负责维护

Redis 集群中内置了 16384 个哈希槽，当需要在Redis 集群中放置一个 key-value 时，redis 先对 key 使用 crc16 算法算出一个结果，然后把结果对 16384 求余数，这样每个 key 都会对应一个编号在 0-16383 之间的哈希槽，redis 会根据节点数量大致均等的将哈希槽映射到不同的节点。

2. redis-cluster投票:容错

(1)投票过程是集群中所有master参与,如果半数以上master节点与其中一个 master节点通信超时(cluster-node-timeout),认为当前master节点挂掉.

(2):什么时候整个集群不可用(cluster_state:fail)?

a:如果集群任意master挂掉,且当前master没有slave.集群进入fail状态,也可以理解成集群的slot映射[0-16383]未完成时进入fail状态. ps : redis-3.0.0.rc1加入cluster-require-full-coverage参数,默认关闭,打开集群兼容部分失败.

b:如果集群超过半数以上master挂掉，无论是否有slave集群进入fail状态.

ps:当集群不可用时,所有对集群的操作做都不可用，收到((error)CLUSTERDOWN The cluster is down)错误。

通过redis集群实现session共享

1. 背景

在web应用中,为了应对大规模访问,必须实现应用的集群部署.要实现集群部署主要需要实现session共享机制,使得多台应用服务器之间会话统一, tomcat等多数主流web服务器都采用了session复制以及实现session的共享. 但问题还是很明显的:

在节点持续增多的情况下,session复制带来的性能损失会快速增加.特别是当session中保存了较大的对象,而且对象变化较快时,性能下降更加显著.这种特性使得web应用的水平扩展受到了限制.

session共享的另一种思路就是把session集中起来管理,首先想到的是采用数据库来集中存储session,但数据库是文件存储相对内存慢了一个数量级,同时这势必加大数据库系统的负担.所以需要一种既速度快又能远程集中存储的服务:redis

开源的spring-session能够解决我们上述的问题。官方网站: <http://projects.spring.io/spring-session/>

Introduction

Spring Session provides an API and implementations for managing a user's session information. It also provides transparent integration with:

- [HttpSession](#) - allows replacing the HttpSession in an application container (i.e. Tomcat) neutral way. Additional features include:
 - **Clustered Sessions** - Spring Session makes it trivial to support [clustered sessions](#) without being tied to an application container specific solution.
 - **Multiple Browser Sessions** - Spring Session supports [managing multiple users' sessions](#) in a single browser instance (i.e. multiple authenticated accounts similar to Google).
 - **RESTful APIs** - Spring Session allows providing session ids in headers to work with [RESTful APIs](#)
- [WebSocket](#) - provides the ability to keep the `HttpSession` alive when receiving WebSocket messages

可参考官方文档进行相应配置

1.1 使用Spring Session之前需要在bm-bom-framework的pom.xml引入相应依赖

```
<dependency>
  <groupId>org.springframework.session</groupId>
  <artifactId>spring-session-data-redis</artifactId>
  <version>1.3.0.RELEASE</version>
  <type>pom</type>
</dependency>
```

1.2 spring session applicationContext-redis.xml配置

引入相应配置文件头部:

```
<beans xmlns="http://www.springframework.org/schema/beans"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xmlns:p="http://www.springframework.org/schema/p"
  xmlns:redis="http://www.springframework.org/schema/redis"
  xmlns:cache="http://www.springframework.org/schema/cache"
  xmlns:c="http://www.springframework.org/schema/c"
  xmlns:util="http://www.springframework.org/schema/util"
  xsi:schemaLocation="http://www.springframework.org/schema/beans
    http://www.springframework.org/schema/beans/spring-beans.xsd
    http://www.springframework.org/schema/redis http://www.springframework.org/schema/redis/spring-redis.xsd
    http://www.springframework.org/schema/cache http://www.springframework.org/schema/cache/spring-
```

cache.xsd

<http://www.springframework.org/schema/util> <http://www.springframework.org/schema/util/spring-util.xsd>

">

```
<util:list id="clusterNodes" list-class="java.util.ArrayList">
  <value>${redis.address1}</value>
  <value>${redis.address2}</value>
  <value>${redis.address3}</value>
  <value>${redis.address4}</value>
  <value>${redis.address5}</value>
  <value>${redis.address6}</value>
</util:list>
```

此配置获取我们本地配置文件中的redis各节点信息.

```
<bean id="redisClusterConfiguration" class="org.springframework.data.redis.connection.RedisClusterConfiguration">
  <constructor-arg ref="clusterNodes"/>
</bean>

<bean id="jedisPoolConfig" class="redis.clients.jedis.JedisPoolConfig" p:maxTotal="${redis.maxTotal}"
  p:maxIdle="${redis.maxIdle}" p:minIdle="${redis.minIdle}" p:maxWaitMillis="${redis.maxWaitMillis}"/>

<bean id="redisConnectionFactory" class="org.springframework.data.redis.connection.jedis.JedisConnectionFactory"
  p:use-pool="true">
  <constructor-arg index="0" ref="redisClusterConfiguration"/>
  <constructor-arg index="1" ref="jedisPoolConfig"/>
</bean>
```

创建redis连接工厂.

```
<bean id="jsonRedisSerializer" class="com.allinpay.bop.bom.cache.GenericJackson2JsonRedisSerializer"/>
<bean id="redisTemplate" class="org.springframework.data.redis.core.RedisTemplate"
  p:connection-factory-ref="redisConnectionFactory" p:defaultSerializer-ref="jsonRedisSerializer"/>

<bean id="cacheManager" class="org.springframework.data.redis.cache.RedisCacheManager" p:usePrefix="true"
  p:defaultExpiration="${redis.defaultExpiration}">
  <constructor-arg ref="redisTemplate"/>
  <!--系统中使用的缓存时间都配置在这里统一管理-->
  <property name="expires">
    <map>
      <entry key="dictionary" value="86400"/>
    </map>
  </property>
</bean>
```

获取到redisTemplate,我们可以通过它在我们项目中与redis打交道.

这里我更换了我们存储到redis的数据的序列化方式.默认的序列化方式是jdk序列化方式,首先确保我们的redis的key是string类型的.

```
public class RedisTemplate<K, V> extends RedisAccessor implements RedisOperations<K, V> {

    private boolean enableTransactionSupport = false;
    private boolean exposeConnection = false;
    private boolean initialized = false;
    private boolean enableDefaultSerializer = true;
    private RedisSerializer<?> defaultSerializer = new JdkSerializationRedisSerializer();
```

这个序列化方法就是Jdk提供的了.首先要求我们要被序列化的类继承自Serializeable接口,然后通过,然后通过Jdk对对象序列化的方法保存。(注:这个序列化保存的对象,即使是个String类型的,在redis控制台,也是看不出来的,因为它保存了一些对象的类型什么的额外信息),对于人类是非可读的

[illegible]

spring data redis提供了几种序列化方式

```

graph TD
    A[Maven: org.springframework.data:spring-data-redis:1.7.1.RELEASE] --> B[spring-data-redis-1.7.1.RELEASE.jar library root]
    B --> C[META-INF]
    C --> D[org.springframework.data.redis]
    D --> E[cache]
    D --> F[config]
    D --> G[connection]
    D --> H[core]
    D --> I[hash]
    D --> J[listener]
    D --> K[repository]
    D --> L[serializer]
    L --> M[GenericJackson2JsonRedisSerializer]
    L --> N[GenericToStringSerializer]
    L --> O[Jackson2JsonRedisSerializer]
    L --> P[JacksonJsonRedisSerializer]
    L --> Q[JdkSerializationRedisSerializer]
    L --> R[OxmSerializer]
    L --> S[RedisSerializer]
    L --> T[SerializationException]
    L --> U[SerializationUtils]
    L --> V[StringRedisSerializer]
  
```

经测验: 从执行时间上来看, JdkSerializationRedisSerializer是最高效的(毕竟是JDK原生的), 但是是序列化的结果字符串是最长的。JSON由于其数据格式的紧凑性, 序列化的长度是最小的, 时间比前者要多一些。而OxmSerialiabler在时间上看是最长的(当时和使用具体的Marshaller有关)。所以个人的选择是倾向使用GenericJackson2JsonRedisSerializer作为POJO的序列化器。

如果使用了GenericJackson2JsonRedisSerializer作为序列化器.项目中发现实体类很多继承了MyBatisBaseModel,序列化时候本类及父类造成很多属性值为null的json.严重增加了我们redis存储空间及值.

```
@Alias(value="Dictionarycategory")
public class Dictionarycategory extends MyBatisBaseModel {
```

原生序列化GenericJackson2JsonRedisSerializer效果如下:

```

192.168.103.28:7115> keys *
1) "dictionary:\com.allinpay.bop.bom.business.baseinfo.business.impl.DictionaryitemBusinessImpl.getDictionaryitemI
istByDicType(Process_Def_Type)"
2) "spring.session.bm:expirations:1484207520000"
192.168.103.28:7115> get "dictionary:\com.allinpay.bop.bom.business.baseinfo.business.impl.DictionaryitemBusinessI
mpl.getDictionaryitemIstByDicType(Process_Def_Type)"
"[{"java.util.ArrayList":{"@class":"com.allinpay.bop.bom.business.baseinfo.model.Dictionaryitem","lookupId":
:null,"lookupName":null,"callJsName":null,"lookupGroupId":null,"orderBy":null,"exact":null,"numPerPage":
:10,"pageNum":1,"pageNumShown":0,"updateTimeBegin":null,"updateTimeEnd":null,"dicType":"Process_Def_Type",
"dicCode":"bom.manage.addBranchAudit","dicName":"\xe6\x96\xb0\xe5\xa2\x9e\xe5\x88\x86\xe5\xba\x97\xe7\x94
\xb3\xe8\xaf\xb7\xe5\xae\xal\xe6\x89\xb9\xe6\xb5\x81\xe7\xa8\x8b","disOrder":["java.math.BigDecimal",1],"updat
eTime":null,"memo":"","branch":"","dicTypeName":null,"@class":"com.allinpay.bop.bom.business.baseinfo.model
.Dictionaryitem","lookupId":null,"lookupName":null,"callJsName":null,"lookupGroupId":null,"orderBy":null,
"exact":null,"numPerPage":10,"pageNum":1,"pageNumShown":0,"updateTimeBegin":null,"updateTimeEnd":null,
"dicType":"Process_Def_Type","dicCode":"bom.manage.addBranchPrepayHeadAudit","dicName":"\xe6\x96\xb0\xe5
\xa2\x9e\xe5\x88\x86\xe5\xba\x97\xe5\x95\x86\xe6\x88\xb7\xe9\xa2\x84\xe4\xbb\x98\xe5\x8d\xal\xe6\x9d\xal\xe7\xba\xbf
\xe5\xae\xal\xe6\x89\xb9","disOrder":["java.math.BigDecimal",2],"updateTime":null,"memo":"","dicTypeN
ame":null,"@class":"com.allinpay.bop.bom.business.baseinfo.model.Dictionaryitem","lookupId":null,"lookupN
ame":null,"callJsName":null,"lookupGroupId":null,"orderBy":null,"exact":null,"numPerPage":10,"pageNum":
1,"pageNumShown":0,"updateTimeBegin":null,"updateTimeEnd":null,"dicType":"Process_Def_Type","dicCode":
"bom.manage.addTerminalAudit","dicName":"\xe6\x96\xb0\xe5\xa2\x9e\xe7\xbb\x88\xe7\xab\xaf\xe6\xb5\x81\xe7\xa8
\x8b","disOrder":["java.math.BigDecimal",3],"updateTime":null,"memo":"","dicTypeName":null,"@class":
"com.allinpay.bop.bom.business.baseinfo.model.Dictionaryitem","lookupId":null,"lookupName":null,"callJsNam
e":null,"lookupGroupId":null,"orderBy":null,"exact":null,"numPerPage":10,"pageNum":1,"pageNumShown":0,
"updateTimeBegin":null,"updateTimeEnd":null,"dicType":"Process_Def_Type","dicCode":"bom.manage.cancelBra
nchAudit","dicName":"\xe5\x88\x86\xe5\xba\x97\xe6\xb3\xa8\xe9\x94\x80\xe6\xb5\x81\xe7\xa8\x8b","disOrder":["
java.math.BigDecimal",4],"updateTime":null,"memo":"","dicTypeName":null,"@class":"com.allinpay.bop.
bom.business.baseinfo.model.Dictionaryitem","lookupId":null,"lookupName":null,"callJsName":null,"lookUpGrou
pid":null,"orderBy":null,"exact":null,"numPerPage":10,"pageNum":1,"pageNumShown":0,"updateTimeBegin":n
ull,"updateTimeEnd":null,"dicType":"Process_Def_Type","dicCode":"bom.manage.cancelETSHHeadAudit","dicName
":"\xe7\xbd\x91\xe5\x85\xb3\xe6\x9d\xal\xe7\xba\xbf\xe6\xb3\xa8\xe9\x94\x80\xe6\x80\xbb\xe5\x85\xac\xe5\x8f\x
e5\xae\xal\xe6\xa0\xb8\xe6\xb5\x81\xe7\xa8\x8b","disOrder":["java.math.BigDecimal",5],"updateTime":null,"m
emo":"","dicTypeName":null,"@class":"com.allinpay.bop.bom.business.baseinfo.model.Dictionaryitem","look
upId":null,"lookupName":null,"callJsName":null,"lookupGroupId":null,"orderBy":null,"exact":null,"numPer

```

因此项目中我重写了此序列化去除了实体类中getter(有些属性属于公共架构比如Page,里面的总数等get是通过Page的属性值算出来的.)以及null.

```

mapper.setVisibility(PropertyAccessor.FIELD, JsonAutoDetect.Visibility.ANY); // auto-detect all member fields
mapper.setVisibility(PropertyAccessor.GETTER, JsonAutoDetect.Visibility.NONE); // and none public getters
mapper.setVisibility(PropertyAccessor.IS_GETTER, JsonAutoDetect.Visibility.NONE); // and none of "is-setters"
mapper.setSerializationInclusion(JsonInclude.Include.NON_NULL);

```

此序列化我模仿项目中Gson.Gson序列化也是不加入getter.通过属性值来序列化.

改造后结果:

```

[root@localhost ~]# redis-cli -h 192.168.101.73 -p 7116 -c
192.168.101.73:7116> get "dictionary:\com.allinpay.bop.bom.business.baseinfo.business.impl.DictionaryitemBusinessI
mpl.getDictionaryitemIstByDicType(Process_Def_Type)"
"[{"java.util.ArrayList":{"@class":"com.allinpay.bop.bom.business.baseinfo.model.Dictionaryitem","numPerPage":
:10,"pageNum":1,"pageNumShown":0,"dicType":"Process_Def_Type","dicCode":"bom.manage.addBranchAudit","dicName":
"\xe6\x96\xb0\xe5\xa2\x9e\xe5\x88\x86\xe5\xba\x97\xe7\x94\xb3\xe8\xaf\xb7\xe5\xae\xal\xe6\x89\xb9\xe6\x
b5\x81\xe7\xa8\x8b","disOrder":["java.math.BigDecimal",1],"@class":"com.allinpay.bop.bom.business.baseinfo
.model.Dictionaryitem","numPerPage":10,"pageNum":1,"pageNumShown":0,"dicType":"Process_Def_Type","dicCode":
"bom.manage.addBranchPrepayHeadAudit","dicName":"\xe6\x96\xb0\xe5\xa2\x9e\xe5\x88\x86\xe5\xba\x97\xe5\x
95\x86\xe6\x88\xb7\xe9\xa2\x84\xe4\xbb\x98\xe5\x8d\xal\xe6\x9d\xal\xe7\xba\xbf\xe5\xae\xal\xe6\x89\xb9",
"disOrder":["java.math.BigDecimal",2]},{"@class":"com.allinpay.bop.bom.business.baseinfo.model.Dictionaryitem",
"numPerPage":10,"pageNum":1,"pageNumShown":0,"dicType":"Process_Def_Type","dicCode":"bom.manage.addTerminal
Audit","dicName":"\xe6\x96\xb0\xe5\xa2\x9e\xe7\xbb\x88\xe7\xab\xaf\xe6\xb5\x81\xe7\xa8\x8b","disOrder":["j
ava.math.BigDecimal",3]},{"@class":"com.allinpay.bop.bom.business.baseinfo.model.Dictionaryitem","numPerPage":
:10,"pageNum":1,"pageNumShown":0,"dicType":"Process_Def_Type","dicCode":"bom.manage.cancelBranchAudit","dicName":
"\xe5\x88\x86\xe5\xba\x97\xe6\xb3\xa8\xe9\x94\x80\xe6\xb5\x81\xe7\xa8\x8b","disOrder":["java.math
.BigDecimal",4]},{"@class":"com.allinpay.bop.bom.business.baseinfo.model.Dictionaryitem","numPerPage":10,"
pageNum":1,"pageNumShown":0,"dicType":"Process_Def_Type","dicCode":"bom.manage.cancelETSHHeadAudit","dicName":
"\xe7\xbd\x91\xe5\x85\xb3\xe6\x9d\xal\xe7\xba\xbf\xe6\xb3\xa8\xe9\x94\x80\xe6\x80\xbb\xe5\x85\xac\xe5\x8f\x
b8\xe5\xae\xal\xe6\xa0\xb8\xe6\xb5\x81\xe7\xa8\x8b","disOrder":["java.math.BigDecimal",5]},{"@class":"com.
allinpay.bop.bom.business.baseinfo.model.Dictionaryitem","numPerPage":10,"pageNum":1,"pageNumShown":0,"dicT
ype":"Process_Def_Type","dicCode":"bom.manage.cancelMchtAudit","dicName":"\xe5\x95\x86\xe6\x88\xb7\xe6\x
3\xa8\xe9\x94\x80\xe5\x88\x86\xe5\x85\xac\xe5\x8f\xb8\xe5\xae\xal\xe6\x89\xb9","disOrder":["java.math.BigDecim
al",6]},{"@class":"com.allinpay.bop.bom.business.baseinfo.model.Dictionaryitem","numPerPage":10,"pageNum":
1,"pageNumShown":0,"dicType":"Process_Def_Type","dicCode":"bom.manage.cancelTLTHeadAudit","dicName":"\xe7
\x80\x9a\xe8\x81\x94\xe9\x80\x9a\xe6\x9d\xal\xe7\xba\xbf\xe6\xb3\xa8\xe9\x94\x80\xe6\x80\xbb\xe5\x85\xac\xe5\x8f
\xe8\xae\xal\xe6\xa0\xb8\xe6\xb5\x81\xe7\xa8\x8b","disOrder":["java.math.BigDecimal",7]},{"@class":"co
m.allinpay.bop.bom.business.baseinfo.model.Dictionaryitem","numPerPage":10,"pageNum":1,"pageNumShown":0,"di
cType":"Process_Def_Type","dicCode":"bom.manage.cancelTerminalAudit","dicName":"\xe7\xbb\x88\xe7\xab\xaf
\xe6\xb3\xa8\xe9\x94\x80\xe6\xb5\x81\xe7\xa8\x8b","disOrder":["java.math.BigDecimal",8]},{"@class":"com.all
inpay.bop.bom.business.baseinfo.model.Dictionaryitem","numPerPage":10,"pageNum":1,"pageNumShown":0,"dicType":
"\Process_Def_Type","dicCode":"bom.manage.editETSHHeadAudit","dicName":"\xe7\xbd\x91\xe5\x85\xb3\xe6\x9d
\xal\xe7\xba\xbf\xe5\x8f\x98\xe6\x9b\xb4\xe5\xae\xal\xe6\xa0\xb8\xe6\xb5\x81\xe7\xa8\x8b","disOrder":["java.math
.BigDecimal",9]},{"@class":"com.allinpay.bop.bom.business.baseinfo.model.Dictionaryitem","numPerPage":10,"
pageNum":1,"pageNumShown":0,"dicType":"Process_Def_Type","dicCode":"bom.manage.editMchtAudit","dicName":
"\xe5\x95\x86\xe6\x88\xb7\xe5\x98\xe6\x9b\xb4\xe5\xae\xal\xe6\x89\xb9\xe6\xb5\x81\xe7\xa8\x8b","disOrder"

```

此时去除了无关序列化值.

1.3 启用spring session缓存

@EnableCaching

@EnableRedisHttpSession(redisNamespace = "bm")

public class RedisCacheConfig {

//缓存前缀拼接包.类.方法

```
public static final String cache_prefix = "#root.targetClassName + '.' + #root.methodName + '(' + ";
```

```
}
```

其中@EnableRedisHttpSession(redisNamespace = "bm")注解创建名称为springSessionRepositoryFilter的过滤器.这个过滤器负责使用spring session替换我们项目中的HttpSession,存储在我们redis服务器.redisNamespace 属性是为了区分不同应用比如bm系统,bm-spe系统等防止冲突.

spring 配置文件中有个关键配置<cache:annotation-driven />,但是配置我们项目中发现spring cache不起作用.放弃此配置改为注解开启.

@EnableCaching 此注解是为了启用spring cache.

1. 添加过滤器

```
<filter>
  <filter-name>springSessionRepositoryFilter</filter-name>
  <filter-class>org.springframework.web.filter.DelegatingFilterProxy</filter-class>
</filter>
<filter-mapping>
  <filter-name>springSessionRepositoryFilter</filter-name>
  <url-pattern>/*</url-pattern>
  <dispatcher>REQUEST</dispatcher>
  <dispatcher>ERROR</dispatcher>
</filter-mapping>
```

注意:此过滤器必须必须作为第一个session过滤器.来确保我们系统中后面过滤器及代码中使用到的httpSession是由spring session管理的,而不是原来的存储在我们各个服务器上的session.

用户登录之后session信息保存到了redis中

```
2) "spring:session:bm:expirations:1484207520000"
```

通过redis集群改造数据字典模块

1. 背景

原有项目缓存是通过构造了全局的map.每次存取缓存都是通过这个map对象.这个map是应用级别的.每个服务器都会保存一份各自的.严重影响了内存使用等.改造通过redis来存储这些缓存数据.

需要涉及到的改造模块大致如下:

整体分3步走:

- 1、基础数据
- 2、商户数据
- 3、CICS交易查询的缓存

项目中原来的清理及加载缓存逻辑如下:

缓存时间

```
private static long detaexpireTime = 60 * 60 * 1000*24;
```

缓存判断失效及加载

```

(read-only)
/**
 * Getter method for property <tt>dictItemMap</tt>.
 *
 * @return property value of dictItemMap
 */
public static Map<String, Map<String, String>> getDictItemMap() {

    if(isNeedReload()){
        System.out.println("缓存需要清理");
        lastQuery = System.currentTimeMillis();
        dictItemMap.clear();
        System.out.println("缓存清理完毕");
    }
    return dictItemMap;
}

/**
 * Getter method for property <tt>dictLayerItemMap</tt>.
 *
 * @return property value of dictLayerItemMap
 */
public static Map<String, Map<String, String>> getDictLayerItemMap() {
    if(isNeedReload()){
        System.out.println("缓存需要清理");
        lastQuery = System.currentTimeMillis();
        dictLayerItemMap.clear();
        System.out.println("缓存清理完毕");
    }
    return dictLayerItemMap;
}

```

此外还有一些其他的缓存相关方法.

定义了这些数据字典等存储到一些map中

```

/** 该Map的key 对应于 字典表的 dicType value 是对应的值得列取*/
private static Map<String, Map<String,String>> dictItemMap = new HashMap<String, Map<String,String>>(); //

/** 该Map的key 对应于 字典表的 dicType value 是对应的值得列取*/
private static Map<String, Map<String,String>> dictLayerItemMap = new HashMap<String, Map<String,String>>(); //

```

Map在Java中构造缓存对象的原理及实现

在实际的项目中，当我们对于某些数据需要经常去访问的时候，我们一般的做法都是将其数据放置到一个缓存(Cache)对象中去，可以达到使用空间节省时间的效果。同时，缓存的使用，也可以减少我们对硬盘或者数据库的频繁访问。达到了比较理想的效果。

这就是为什么我们在java中需要对某个对象设置缓存的原因（也即原理）。在Java中最常见的一种实现缓存的方式就是使用Map，而缓存的实现则主要分为三大步骤：

第一：先到缓存里面查找是否有我们需要的对象，若是存在，则直接返回。

第二：若是缓存里面没有该对象，则将其数据添加到缓存中去。以备下次使用。

第三：如果找到了相应的数据，或者是创建了相应的数据，那就直接使用这个数据。

然而，java中Map对象的key设置为String类型，也是为了查找的方便。

不同缓存的优缺点：

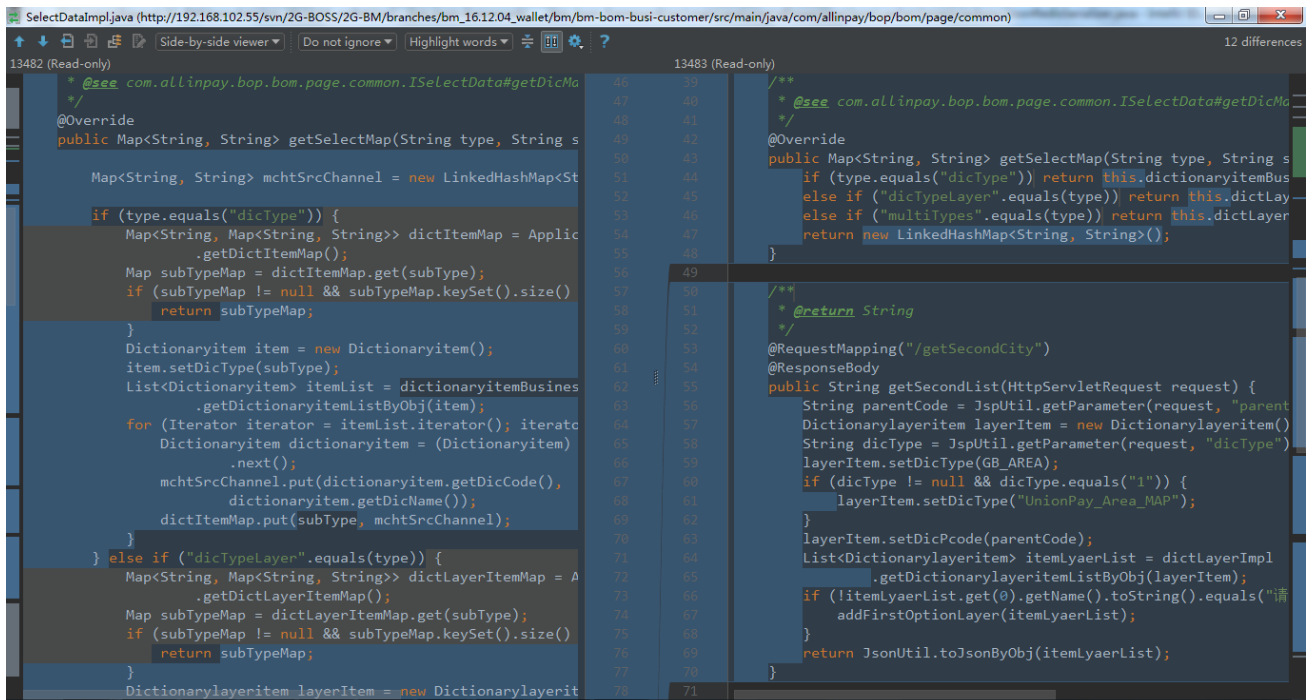
1. 使用局部缓存要注意内存溢出

一般局部缓存使用Map来实现，很容易无限制的往Map中填入数据，导致 内存溢出。

2. 项目规模小的时候,Map缓存是在本地内存里,此时并无多少影响.

服务器端内存不足的话需要采用分布式缓存. 现在的缓存框架可做分布式, 而且可缓存的数据绝对比map大, 部分缓存框架有采用压缩或文件缓存达到降低内存的占用.

3. 缓存代码和业务代码耦合度太高, 如下面的左边例子, 方法中有了太多缓存的逻辑, 不便于维护和变更



缓存的存储这块写的比较死, 不能灵活的切换为使用第三方的缓存模块

3. 远程集中缓存要考虑远程调用开销和缓存对象的序列化和反序列化开销。

远程缓存就需要通过远程调用缓存服务器来实现, 远程调用开销比较大, 所以缓存对象设置的要合理, 避免获取一个完成对象的时候多次访问缓存信息。又由于序列化和反序列化的开销, 缓存对象要尽可能的小。

4. 远程集中式缓存需要考虑单点问题。如果缓存服务器宕机, 所有的对缓存访问都失效。需要考虑备份的缓存服务器。

.....

目前我们缓存数据较少.暂不考虑太多.

我们怎么使用Redis?

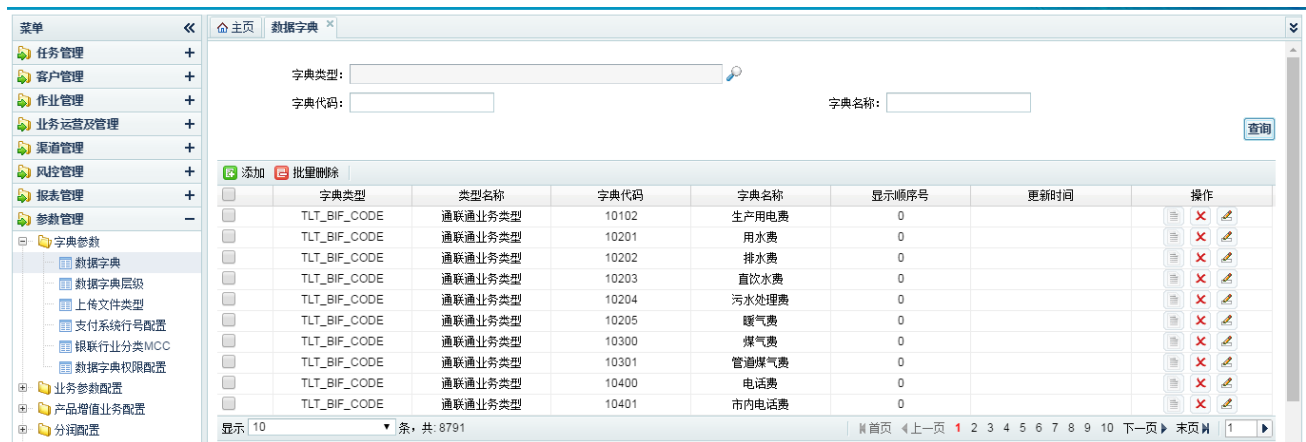
使用redis缓存时候我的思考:

1. 对于数据缓存不是所有东西都缓存到redis就是好的, 而是要针对一些改动不大或者访问率大的数据进行缓存来减少关系型数据库的压力。
2. 不要试图在拦截器或者过滤器中判断是否有缓存的存在, 因为每个请求(不管该请求对应的方法是否做了缓存)它都会去redis中请求数据并判断, 这样会浪费一定的内存资源跟响应时间。所以应该针对需要缓存的方法进行判断。
3. 使用SpringCache, 缓存添加在事务层, 发现方法内部 互相调用的时候, 并不会自动触发Spring的缓存操作

官方也不建议this这种内部调用.禁止缓存方法同类this调用.事务等AOP思想的基本不要this调用最好调用下一层.

4. 分页不做缓存.条件查询除了重要的不做缓存.特别是分页加条件查询的.每一次不通参数有不通的查询结果.可能性太多.目前我们服务器两台4G.怕撑不住

ps:



.....其他

总而言之针对:热点数据缓冲.读的频繁,写的少的数据.session的共享.

在项目中我们没有使用redis一些复杂功能比如:1.图片的信息、2.登录后的用户信息、3.全局超时锁、4.验证码、5.抢红包、6.秒杀.....

目前非常满足80/20法则(又称为:帕雷托法则),查询的业务量远远多于写入的业务量,为了提高TPS,降低对数据库的访问。我们也采取常规的做法,选用redis进行缓存常用业务数据。

目前还在雏形孵化阶段,没有考虑太多关于分布式以及高可用的方案,对redis的使用很粗糙,在团队内可能熟悉redis的Developer不多。

依我看来目前spring cache基本满足我们需要.如果后期有复杂功能考虑定制客户端或自己定义aop注解进行缓存.

项目中使用spring cache

@EnableCaching

@EnableRedisHttpSession(redisNamespace = "bm")

public class RedisCacheConfig {

//缓存前缀拼接包.类.方法

public static final String cache_prefix = "#root.targetClass.name + '.' + #root.methodName + '(' + ";

}

spring 配置文件中有个关键配置<cache:annotation-driven />,但是配置我们项目中发现spring cache不起作用.放弃此配置改为注解开启.

@EnableCaching 此注解是为了启用spring cache.

上面提到了@EnableCaching 为了代替<cache:annotation-driven />要不可以保持系统大多数使用xml配置方法.

@EnableRedisHttpSession这个注解和spring session官网XML配置一样由于spring cache采用了注解.我这里也用这个注解代替如下的类:

```
<bean  
class="org.springframework.session.data.redis.config.annotation.web.http.RedisHttpSessionConfigur  
ation"/>
```

2

spring cache大致有几个注解:

1. @Cacheable

2. @CachePut(如果你的service方法查询结果都是单个记录或者说都是根据id等单属性可以考虑)

3. @CacheEvict

目前我们项目中一个service有list和单个的.我目前只用了@Cacheable和@CacheEvict.

关于spring cache这几个注解可以自行百度了解.它还是比较强大的比如支持SPEL表达式等.

参考连接http://blog.csdn.net/zhu_tianwei/article/details/49072311

项目改造

```
@CacheEvict(value = "dictionary", allEntries = true)  
public String add(Dictionaryitem dictionaryitem) {  
    return dictionaryitemServiceImpl.add(dictionaryitem);  
}
```

@CacheEvict使用范围: 增加 修改 删除的使用

这几个注解都有一个value属性.相当于在redis开辟了一个命名空间一样.相关的都存到这一块.

目前期望多定义区域(区域尽可能小)每次增删改时候@CacheEvict可以让redis一次性失效的key少点,因为一次性过期大量的key对redis是一个不小的冲击.

另外尽量不要一个key缓存的值过大.就如热点存储或者说负载不均衡导致一个redis节点的存储内存过大.

```
@Cacheable(value = "dictionary", key = RedisCacheConfig.cache_prefix + " #dicType ")  
public List<Dictionaryitem> getDictionaryItemListByDicType(String dicType) {  
    Dictionaryitem dictionaryitem = new Dictionaryitem();  
    dictionaryitem.setDicType(dicType);  
    return dictionaryitemServiceImpl.getDictionaryItemListByObj(dictionaryitem);  
}
```

这个注解是缓存我们数据到redis中.包括缓存的区域.以及我们缓存key

这个key很重要.我起初用的方法名字加参数作为key.发现相同模块service有些方法名参数都一样造成了key重复.这是很危险的.为了解决这一问题.我目前加入了自动拼接前缀:

//缓存前缀拼接包.类.方法

```
public static final String cache_prefix = "#root.targetClass.name + '.' +  
#root.methodName + '(' + ";
```

自动拼接了包.类.方法.当然我们bm项目有不通module块.如果有不同的module有可能包名类名方法参数一样.这概率太小,可以考虑迁移代码到一个模块或者起不通的value(区域)

当然这只是折中方案.(key比较长.redis做hash可能慢点.原则上key尽可能短最多10-20位之间把预估)

拼接之后参数redis展示如下:

```

1) "dictionary":"com.allinpay.bop.bom.business.baseinfo.business.impl.DictionaryitemBusinessImpl.getDictionaryitemL
istByDicType(Process_Def_Type\""
2) "spring:session:bm:expirations:184207520000"
192.168.103.28:7115> get "dictionary:"com.allinpay.bop.bom.business.baseinfo.business.impl.DictionaryitemBusinessI
mpl.getDictionaryitemListByDicType(Process_Def_Type\""
["java.util.ArrayList", [{"@class":"com.allinpay.bop.bom.business.baseinfo.model.Dictionaryitem", "lookupId"
:null, "lookupName":null, "callJsName":null, "lookupGroupId":null, "orderBy":null, "exact":null, "numPerPage"
:10, "pageNum":1, "pageNumShown":0, "updateTimeBegin":null, "updateTimeEnd":null, "dicType":"Process_Def_Type
", "dicCode":"bom.manage.addBranchAudit", "dicName":"\xe6\x96\xb0\xe5\xa2\x9e\xe5\x88\x86\xe5\xba\x97\xe7\x94
\xb3\xe8\xaf\xb7\xe5\xae\xa1\xe6\x89\xb9\xe6\xb5\x81\xe7\xa8\x8b", "dispOrder":["java.math.BigDecimal", 1], "upd
ateTime":null, "memo":"branch", "dicTypeName":null}, {"@class":"com.allinpay.bop.bom.business.baseinfo.model
.Dictionaryitem", "lookupId":null, "lookupName":null, "callJsName":null, "lookupGroupId":null, "orderBy":null
, "exact":null, "numPerPage":10, "pageNum":1, "pageNumShown":0, "updateTimeBegin":null, "updateTimeEnd":null,
"dicType":"Process_Def_Type", "dicCode":"bom.manage.addBranchAudit", "dicName":"\xe6\x96\xb0\xe5\xa2\x9e\xe5\x88\x86\xe5\xba\x97\xe7\x94
"}]]]

```

key都是string可以直接get key获取

存储到redis时候key很重要要保持唯一。

如果有其他复杂功能spring cache无法应对可以自己调用jedis的api封装.但是务必保证key唯一。

如:

```

@Component
public class RedisOperator {

    private Logger logger = LoggerFactory.getLogger(RedisOperator.class);

    @Autowired
    private StringRedisTemplate stringRedisTemplate;

    /**
     * 与spring cache 区别开加前缀
     */
    public static final String KEY_PREFIX_VALUE = "cache_value:";
    public static final String KEY_PREFIX_SET = "cache_set:";
    public static final String KEY_PREFIX_LIST = "cache_list:";

    /**cache:value=====*/

    /**
     * 缓存value操作
     */
    public boolean cacheValue(String k, String v, long time) {

        String key = KEY_PREFIX_VALUE + k;

        try {
            ValueOperations<String, String> valueOperations = stringRedisTemplate.opsForValue();

            valueOperations.set(key, v);

            if (time > 0) stringRedisTemplate.expire(key, time, TimeUnit.SECONDS);

            return true;
        } catch (Throwable t) {
            logger.error("缓存[" + key + "]失败, value[" + v + "]", t);
        }
    }
}

```

可以特殊模块起后缀保证唯一。